

Demo_material

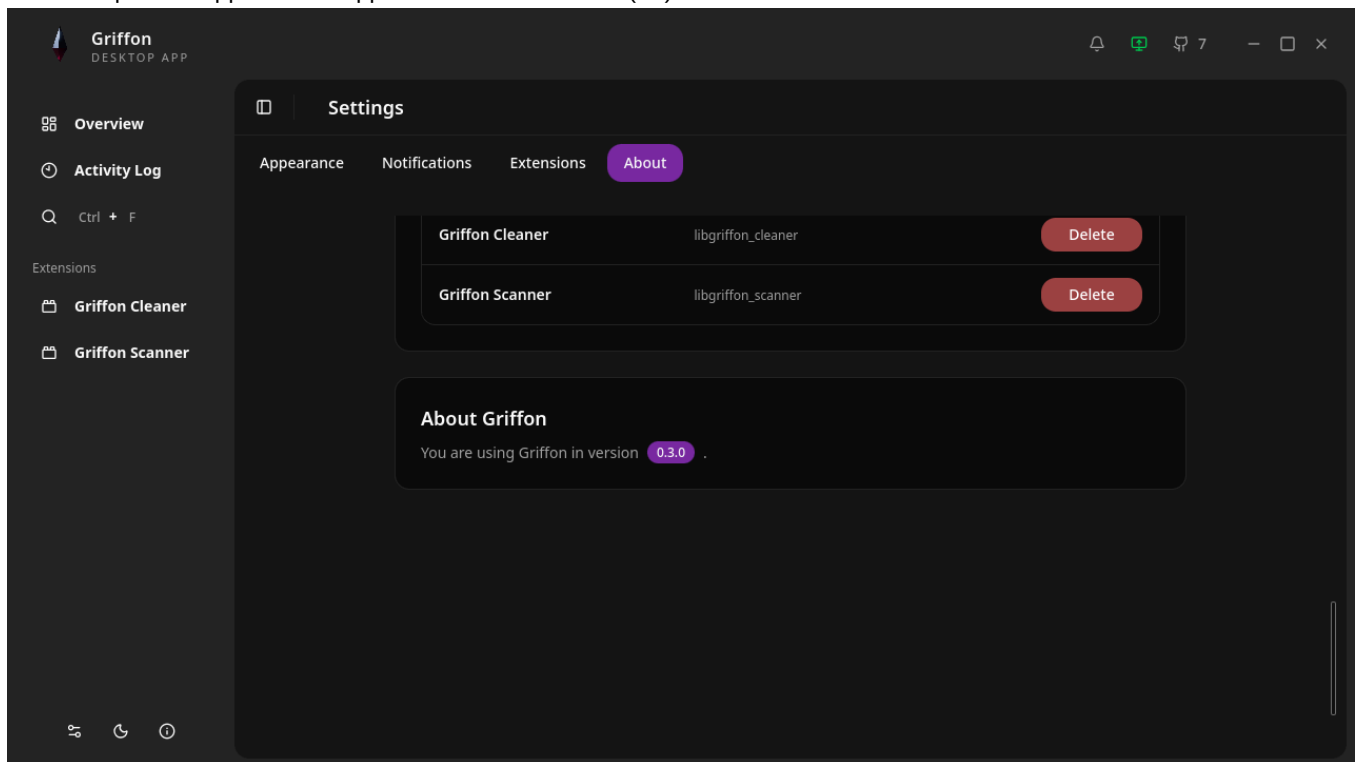
Demonstration Prerequisites

- **Time limit:** 20 minutes maximum. Keep the pace dynamic.
- **Requirements to prep beforehand:**
 - Zip file for installing the Cleaner plugin.
 - Zip file for installing the Scanner plugin.
 - Zip file for installing the Docker plugin.
 - A "dummy malware" zip file downloaded and ready in a specific folder to test the scanner.

Act 1: Overall Presentation of the App

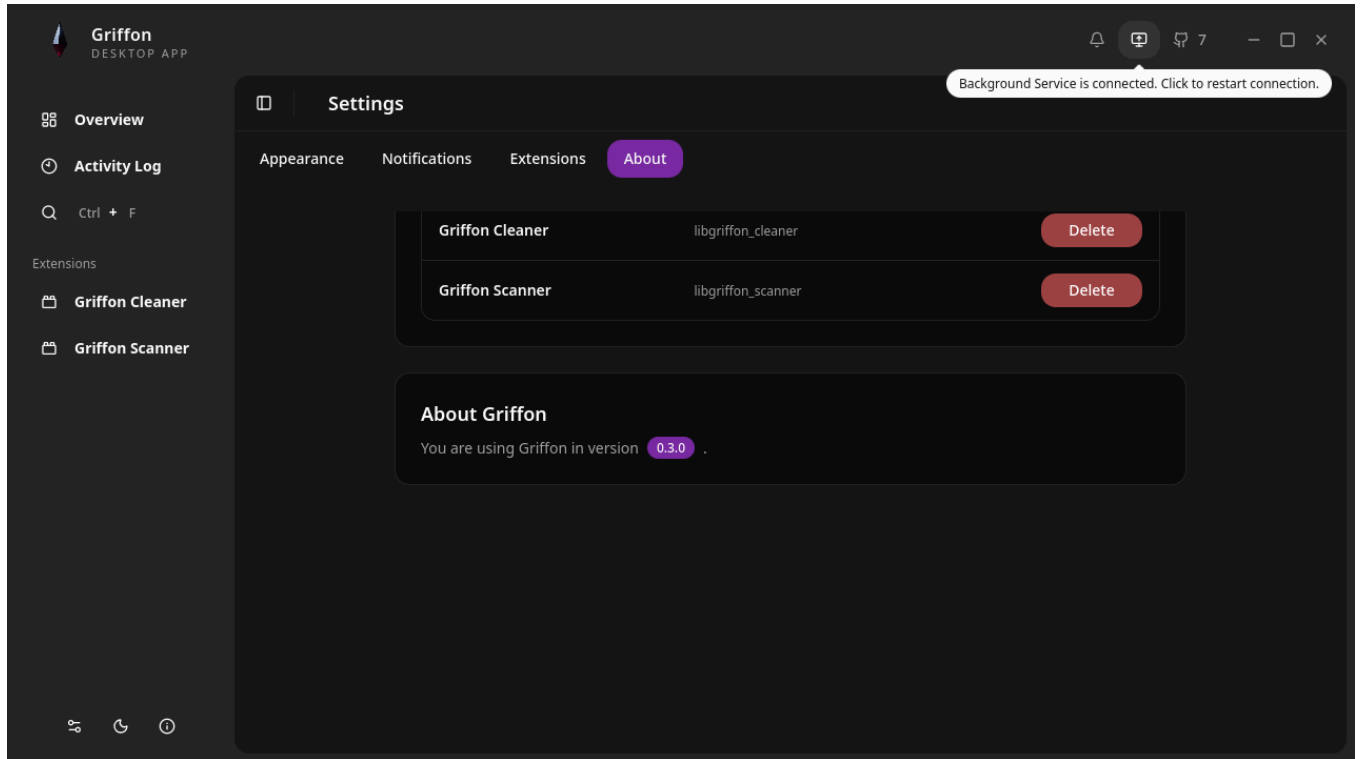
Goal: Prove stability and establish the modular architecture.

- **Action:** Open the app from the application menu on Linux (**F1**).

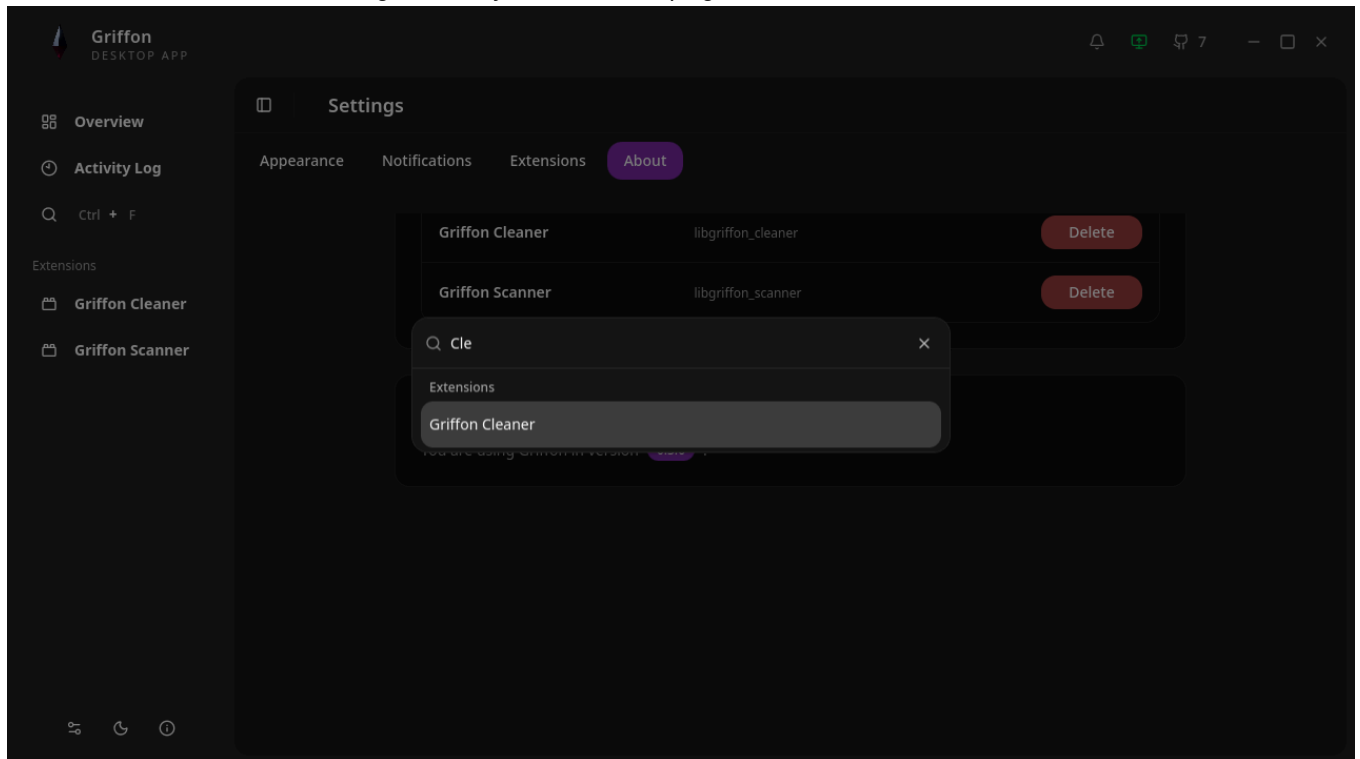


- **Talking point:** The dashboard is displayed. Explain that the app acts as a unified hub where users only install the plugins they actually need.
- **Action:** Show the dynamic list of loaded plugins on the dashboard (**F2**), along with quick access to settings and logs.

- **Action:** Point out the connection status with the Rust background service (the little icon in the top right corner).



- **Action:** Use the search bar to navigate directly to the Cleaner plugin.

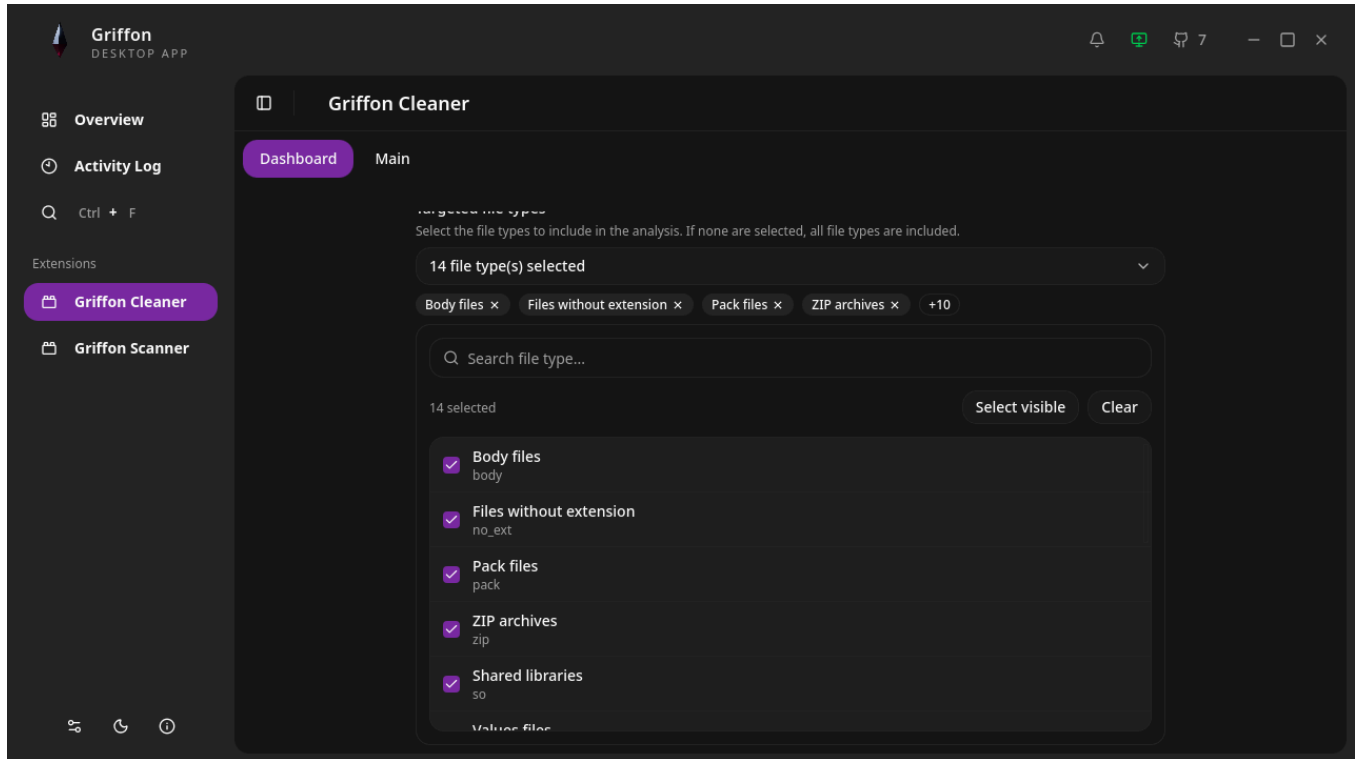


Act 2: Cleaner Plugin

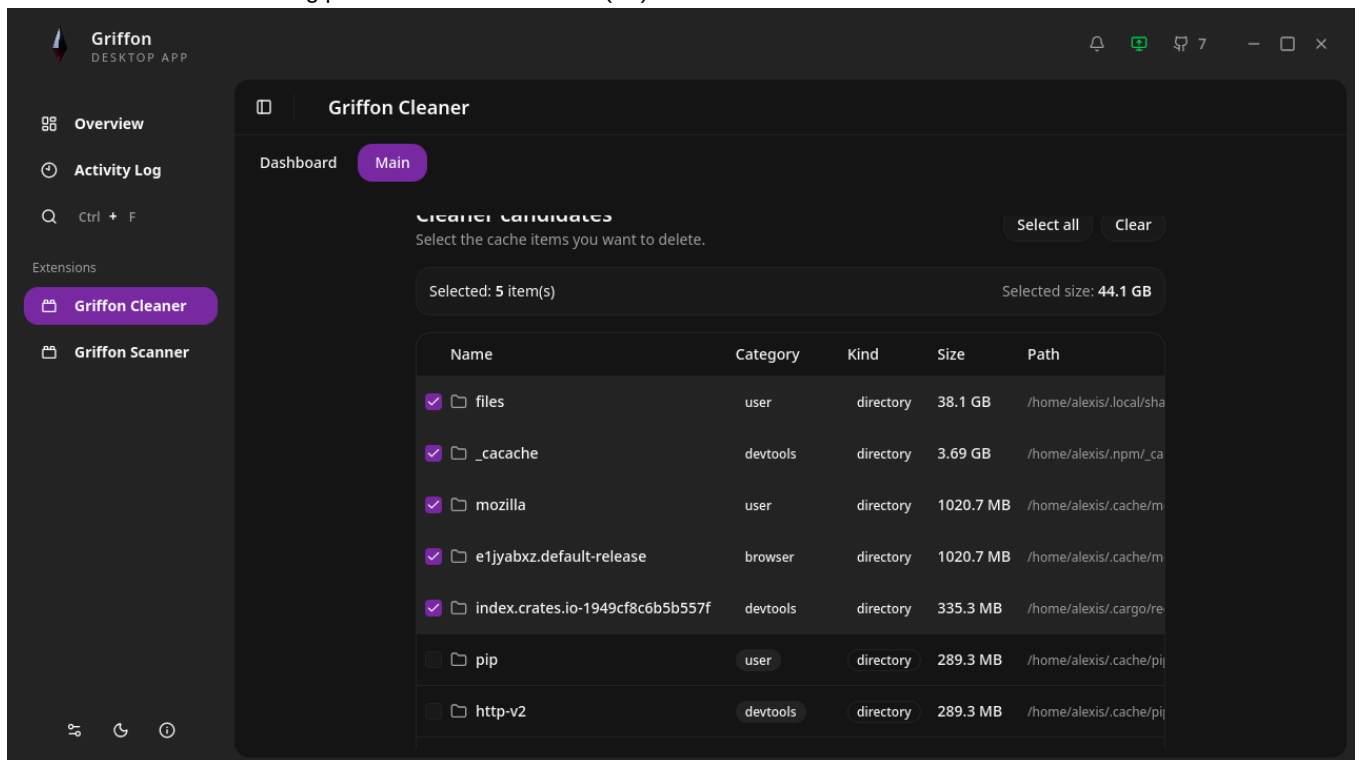
Goal: Demonstrate a complete, simple utility workflow.

- **Talking point:** "The most basic plugin we have is the System Cleaner. Its goal is to clear all temporary and useless files to help speed up the computer."

- **Action:** Select the specific types of data/files you want to clear (**F3**).



- **Action:** Run the search/analysis (**F4**).
- **Action:** When the results window appears, review the detected files with the jury to show the data is accurate (**F5**). Select all files.
- **Action:** Execute the cleaning process to delete the files (**F6**).

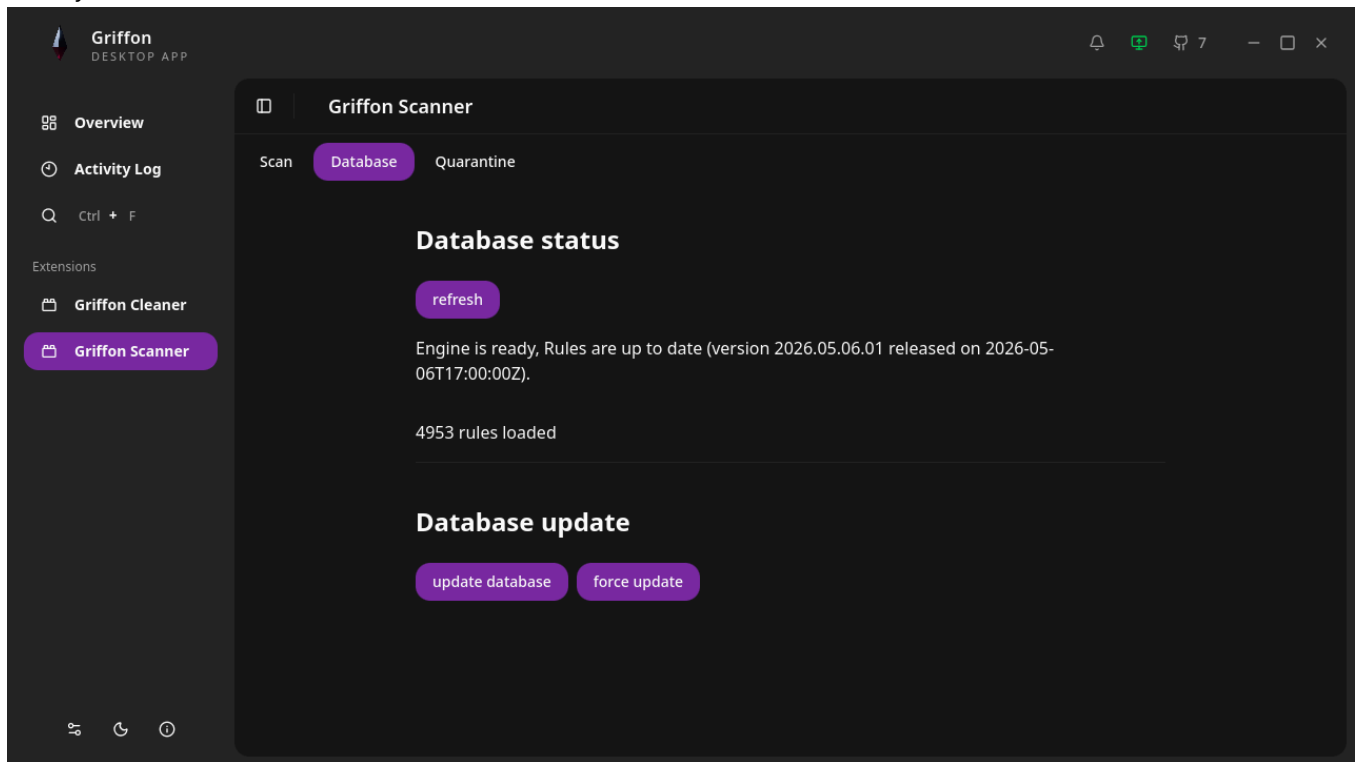


Act 3: Scanner Plugin

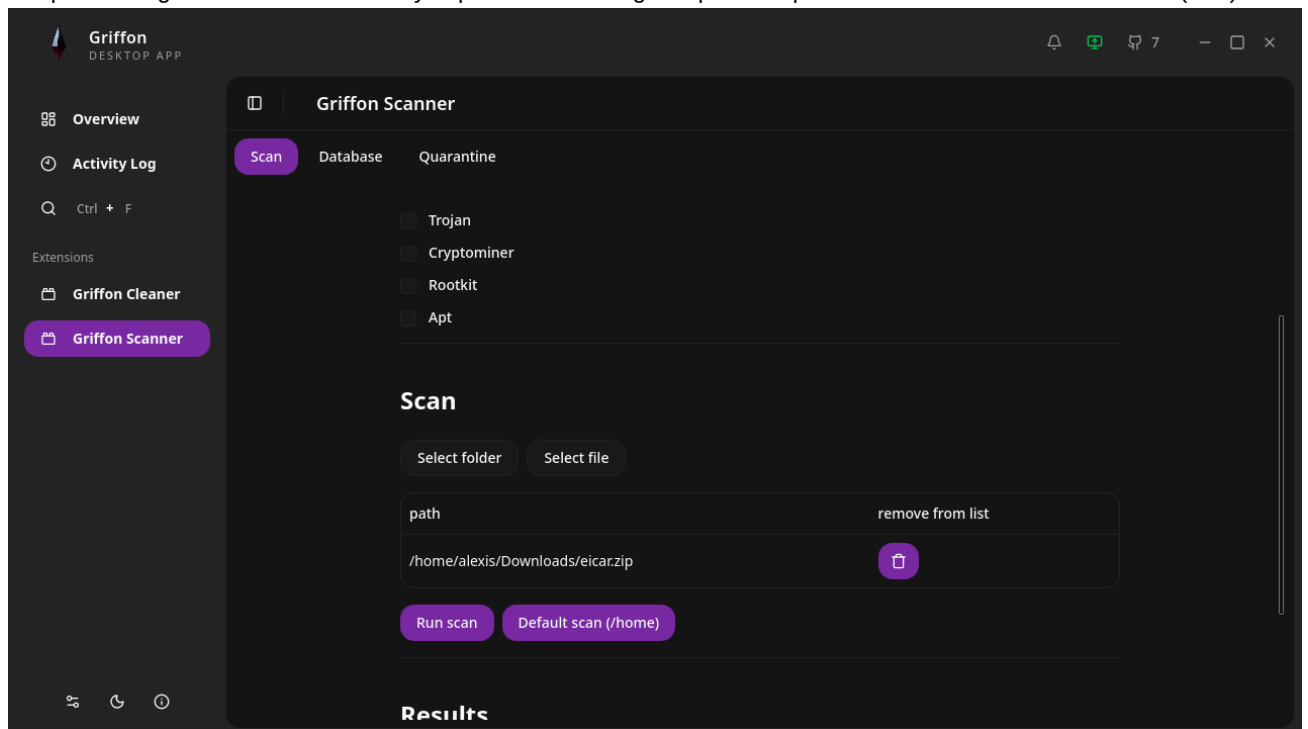
Goal: Showcase the core security engine, advanced configuration, and background processing.

- **Talking point:** "Our main tool is the Scanner. It is useful for any type of user—for instance, if you download a weird file online, a piece of code, or a suspicious zip archive."

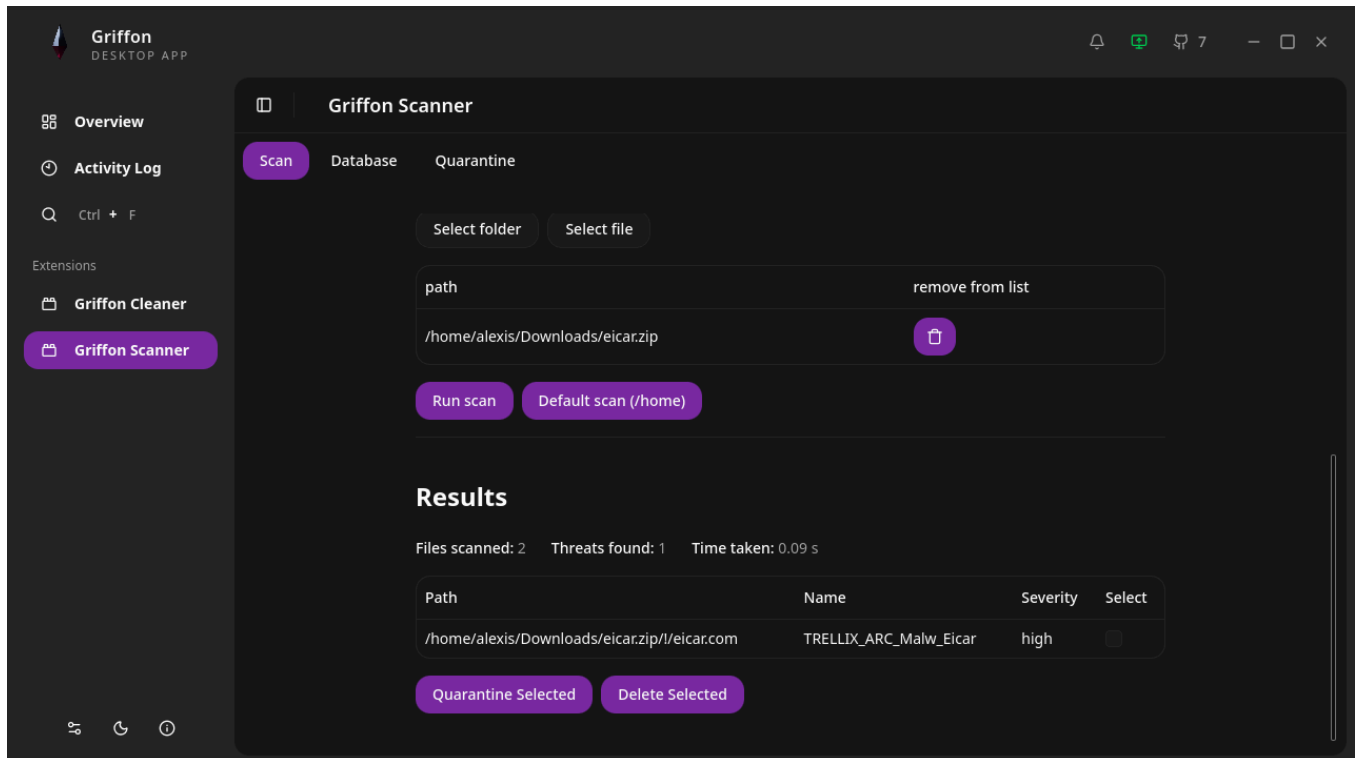
- **Action:** First, open the database tab and press "Update" to show the jury that the database is up-to-date and YARA rules are actively loaded.



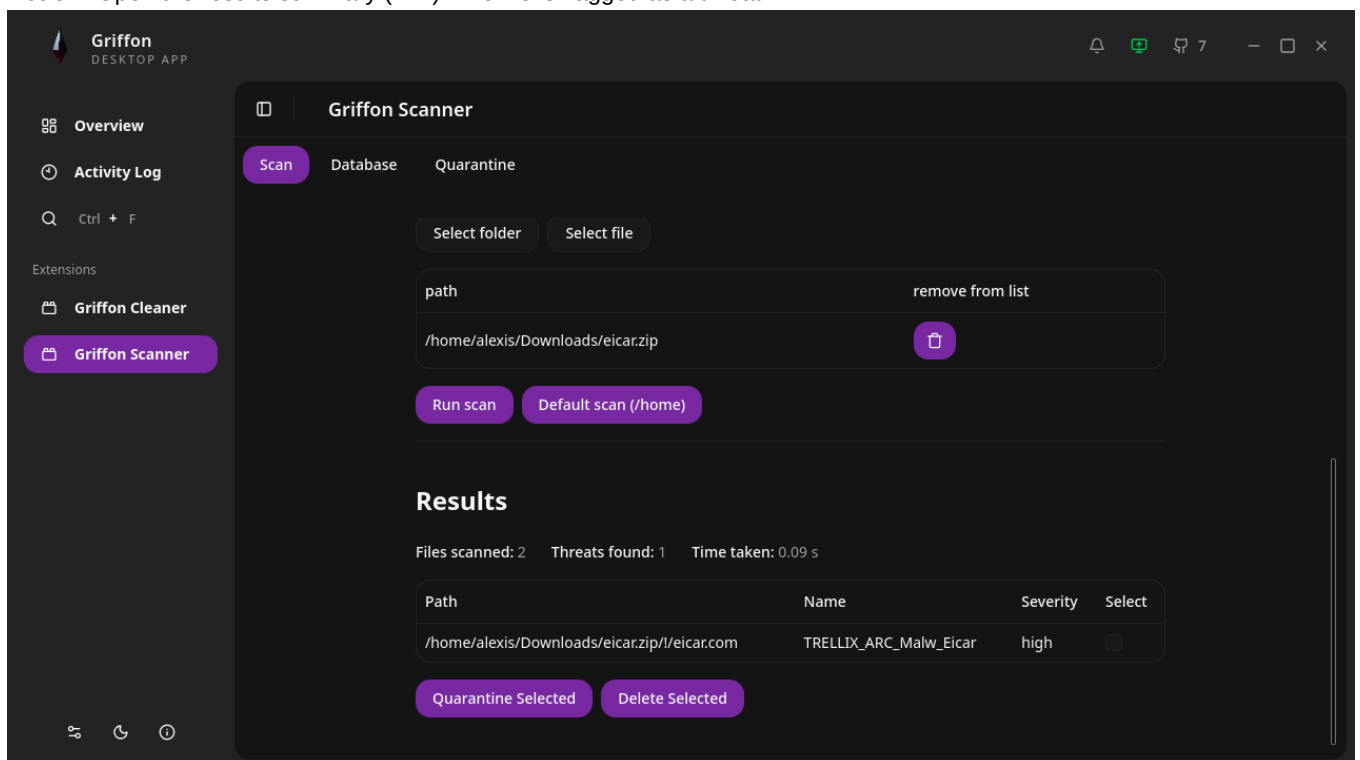
- **Action:** Target the specific dummy malware zip file you downloaded earlier (F7).
- **Action:** Configure the scan:
 - Select "Include archived files" (F13).
 - Select "All threats" to apply threat filters (F9).
 - Keep threading set to "Auto" and briefly explain how the engine optimizes performance based on thread count (F10).



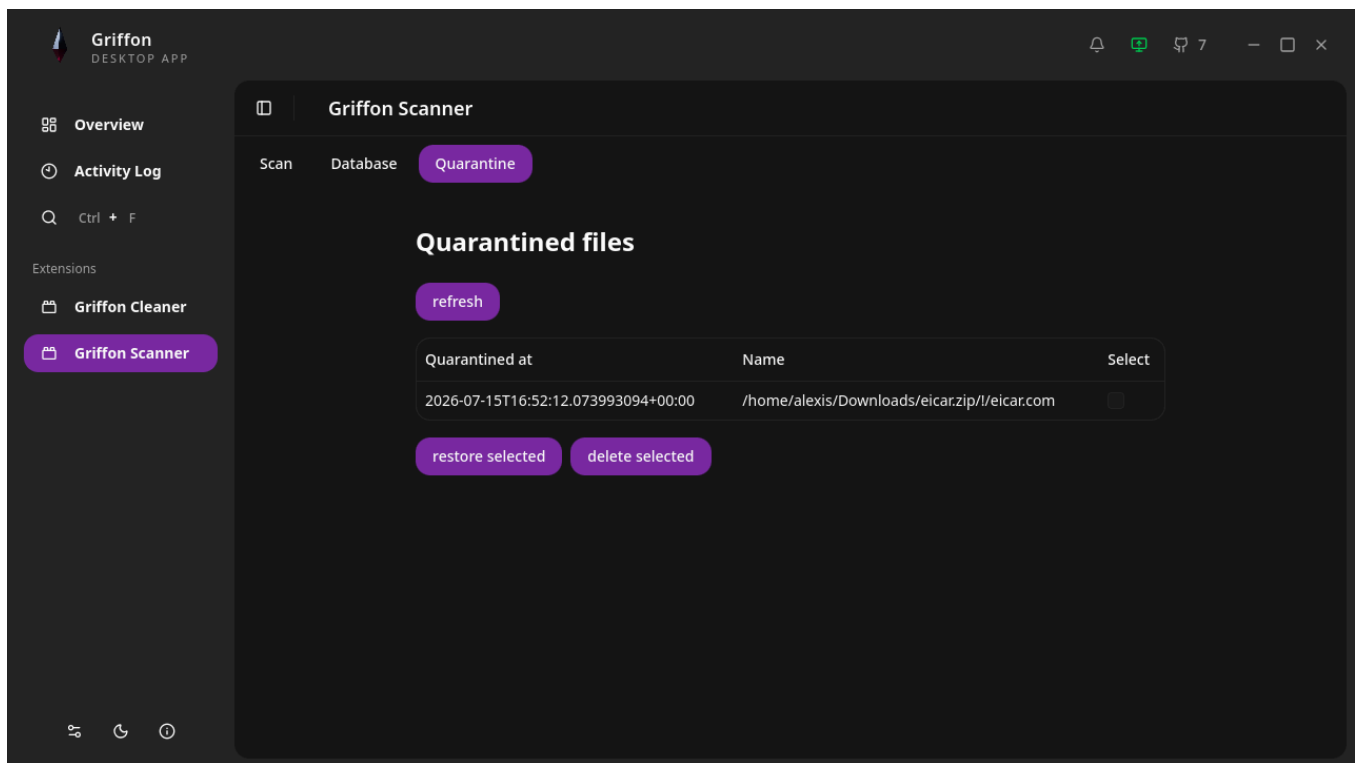
- **Action:** Run the scan.



- **Talking point:** While it runs, talk about the notification system. Explain that when any task finishes (foreground or background), the user is alerted.
- **Action:** The scan finishes and triggers a system notification (**F15**).
- **Action:** Open the results summary (**F11**). The file is flagged as a threat.



- **Action:** Select the threat and confine it to quarantine (**F12**).
- **Action:** Open the Quarantine tab to show the isolated file. Explain that from here, the user can permanently remove or restore the file (**F12**).

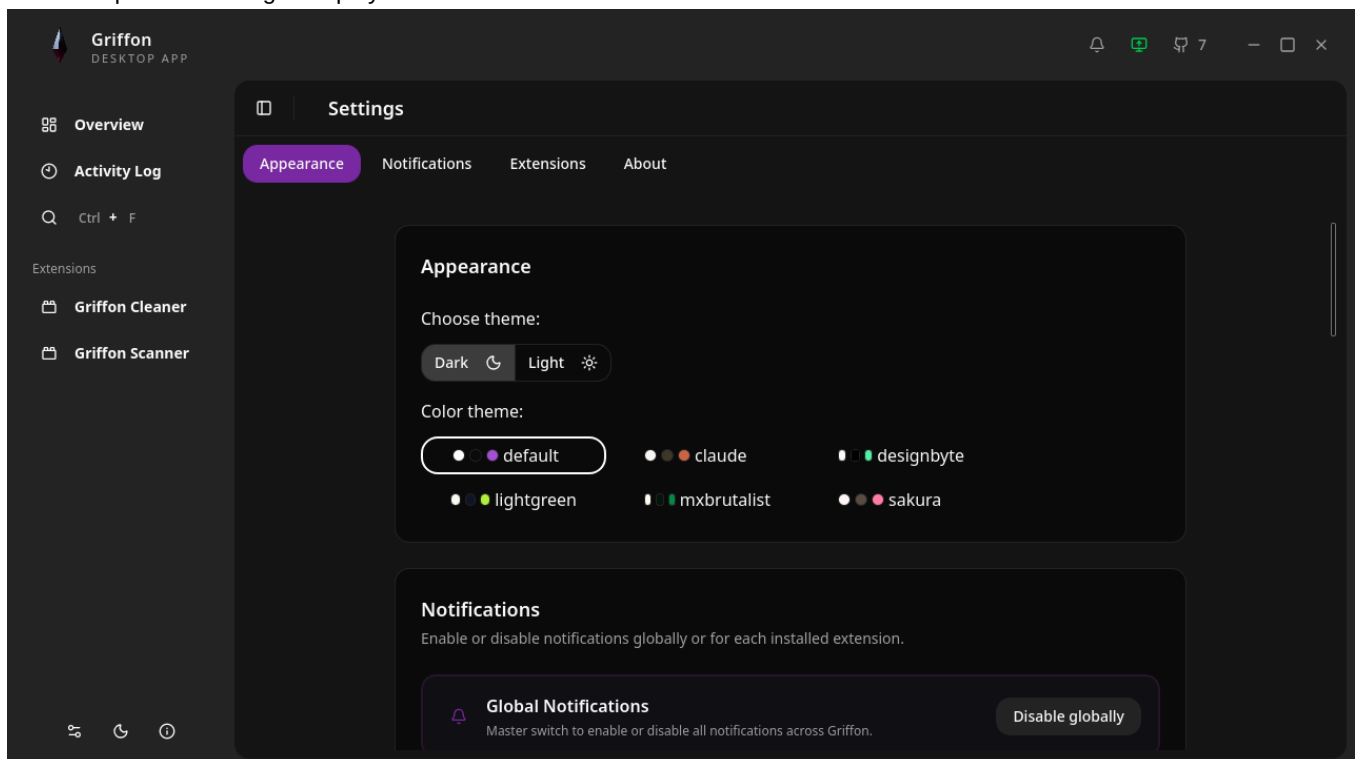


- **Action:** Finally, explain that users don't have to manually configure everything. Run a "Default Analysis" (e.g., scanning the /home directory) to show that the scanner is smart enough to quickly skip safe files using predefined settings (**F8**).

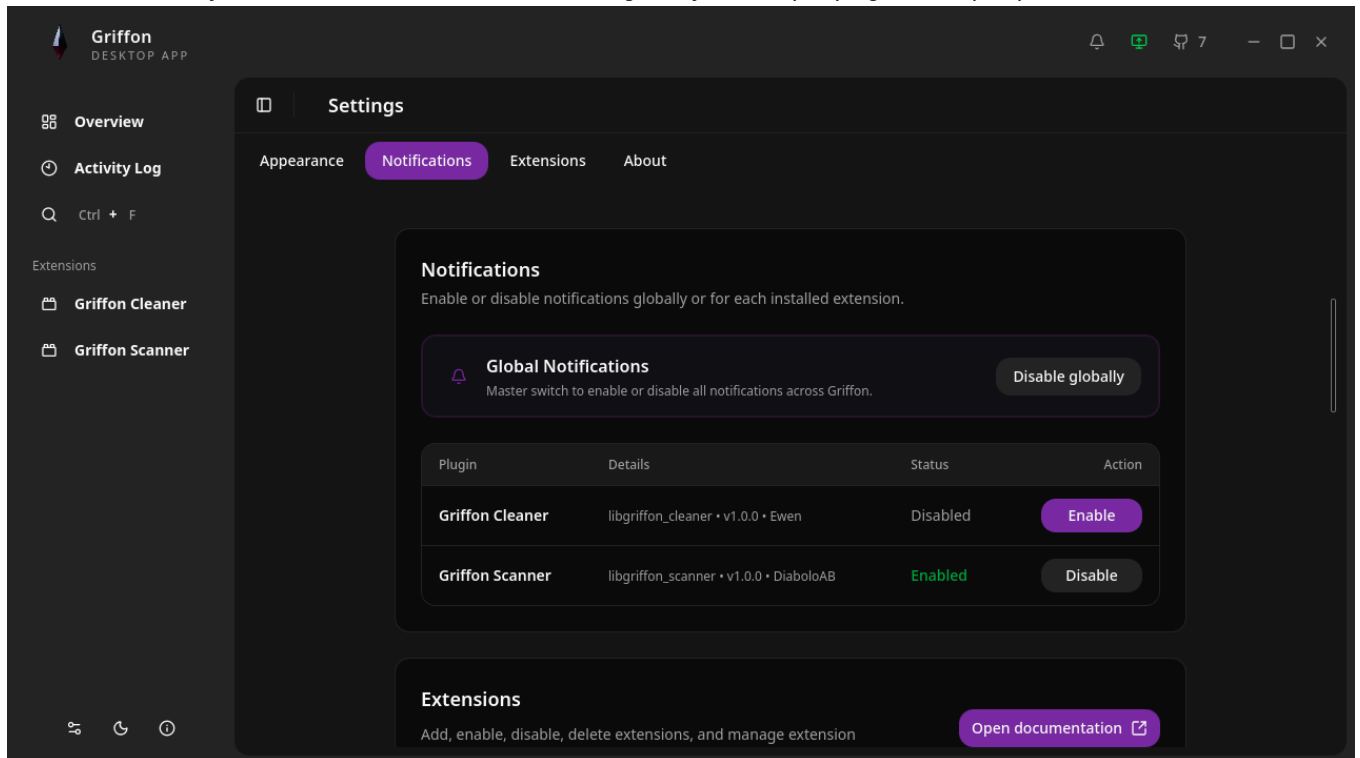
Act 4: Modularity and Auditability

Goal: Prove user control and highlight the developer ecosystem.

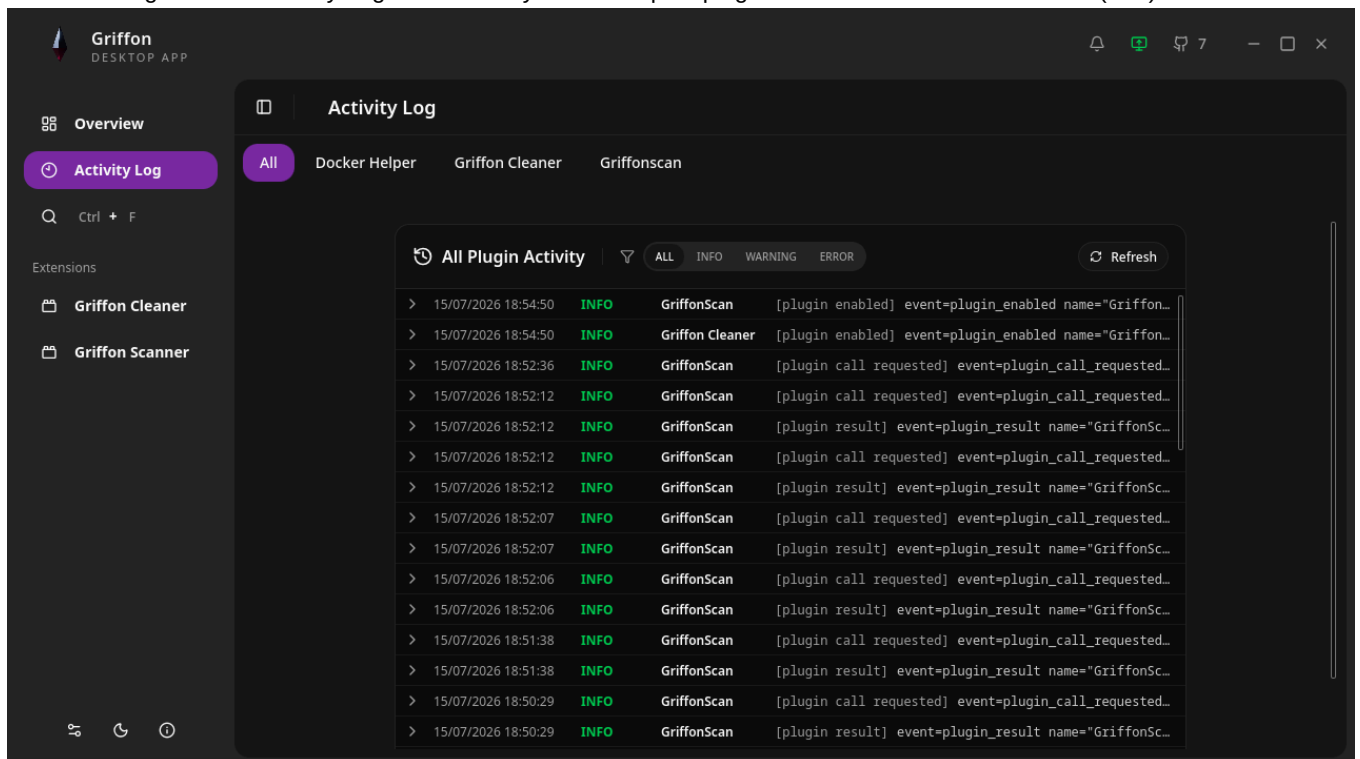
- **Talking point:** "Our app is highly configurable. We give control back to our users to keep them safe and comfortable."
- **Action:** Open the Settings. Display the different UI themes.



- **Action:** Show that you can enable or disable notifications globally or on a per-plugin basis (F18).

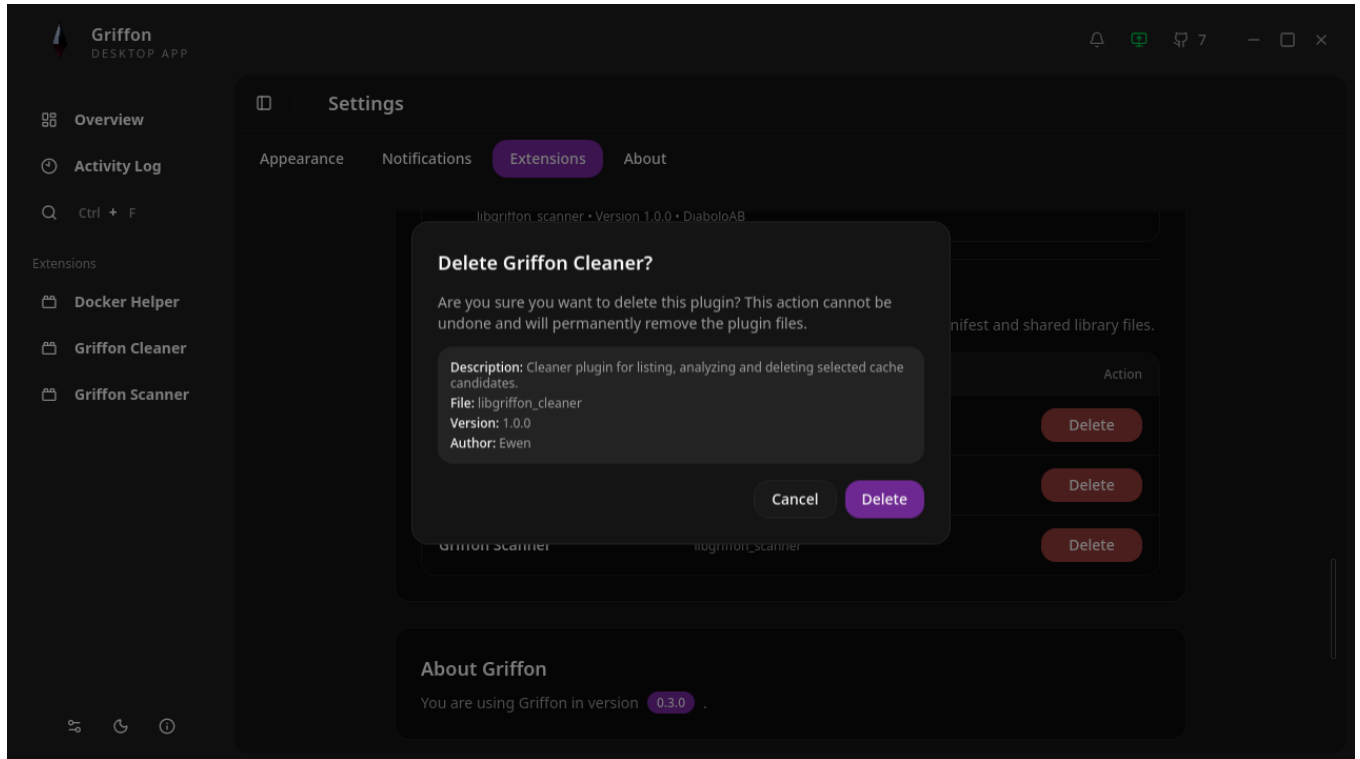


- **Action:** Navigate to the History/Logs tab to briefly show that past plugin tasks are saved and auditable (F14).

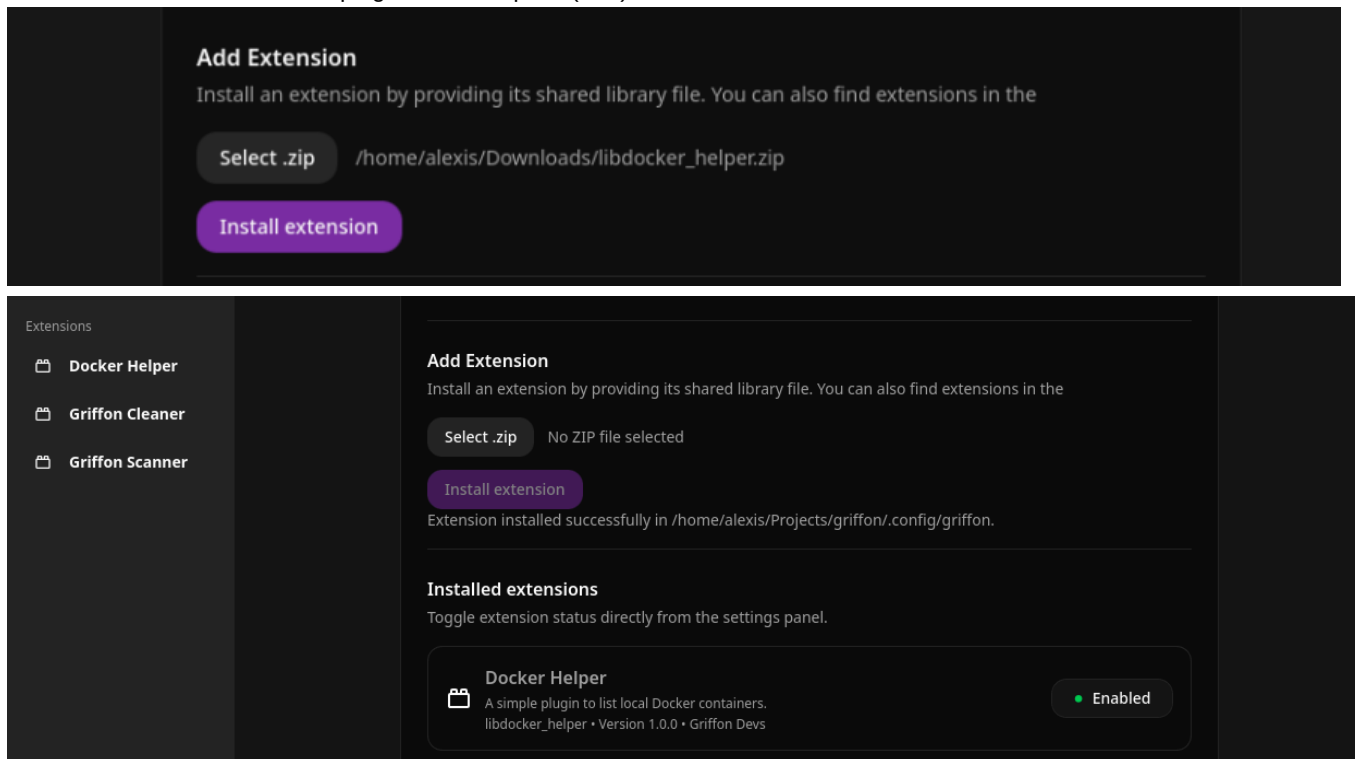


- **Action:** Show the plugin manager. Explain that you can disable any plugin so it no longer loads in the background (F16).

- **Action:** Demonstrate the plugin lifecycle by deleting the Cleaner plugin (**F17**).



- **Talking point:** "Plugins can be made by the community, and you can load them easily."
- **Action:** Install/load the Docker plugin from its zip file (**F17**).



- **Action:** Open the Docker plugin and do a very quick presentation.
- **Closing statement:** "Here is a plugin made directly using our technical documentation (**F19**). It simply parses system commands to display your active Docker containers, a simple tool to help developers keep track of their work, proving the

infinite expandability of the Griffon engine."

Griffon

Install Griffon

Docs

Blog

Introduction

Installation & Releases

Plugin Development

Plugin Architecture

The UI Manifest & Store

Interactions

The Rust Backend

Packaging & Loading

Technologies choice

Team documents

Contributing

Home

Plugin Development

Plugin Architecture

Building a Griffon Plugin

Welcome to the Griffon plugin developer documentation! This section will walk you through everything you need to build, package, and load your own plugin.

The two-file model

A Griffon plugin is made of exactly **two files**:

File	Role
<code>.so</code> (dynamic library)	A Rust <code>cdyLib</code> that implements your plugin's logic. It exposes a small, stable interface that Griffon calls into.
<code>.toml</code> (manifest)	A configuration file that tells Griffon what UI to render, what data to keep track of, and which Rust function to call for each user interaction.

To install a plugin, you **zip these two files together** and load the archive from the Griffon settings menu. There is no separate installer, build step on the host side, or registration process — Griffon reads the manifest, builds the UI from it, and dynamically loads the library.

ⓘ FUTURE TOOLING

Writing the `.toml` manifest by hand is currently the most time-consuming part of building a plugin. We are actively working on tooling that will let most of this configuration be generated directly from your Rust library, so that in the future you'll only need to write a minimal manifest — or none at all. That tooling doesn't exist yet, so for now the manifest has to be written manually. This guide covers the manual process as it stands today.

The Griffon mental model

The two-file model

The Griffon mental model

What each file is responsible for